Manual Implementation of a Multilayer Perceptron: From Forward Pass to Advanced Optimizers

Nazakat Ali
Roll No: 25I-3029

Abstract

This report presents a complete manual and programmatic implementation of a Multilayer Perceptron (MLP) completed as part of the MVC Project for the Artificial Neural Networks course. The project builds an MLP step by step — beginning with the forward pass on a three-sample student performance dataset, progressing through Mean Squared Error (MSE) loss calculation, implementing backpropagation via the chain rule of calculus using the delta method, and applying gradient descent weight updates. Batch gradient descent, mini-batch gradient descent with shuffling across three epochs, and advanced optimizers including SGD with Momentum and Nesterov Accelerated Gradient (NAG) are studied and compared. Assigned weights (w1=0.33, w2=0.38, w3=0.03, w4=0.34, w5=0.38, w6=0.74, w7=0.08, w8=0.18, w9=0.42, w10=0.33, $b^{(1)}$=0.16, $b^{(2)}$=0.04) are used throughout all manual tasks. The Python implementation in Task 7 scales the identical architecture to the MNIST handwritten digit dataset (70,000 images), achieving competitive classification accuracy using only NumPy — no deep learning frameworks. All manual calculations follow the step-by-step procedure outlined in the project PDF, with results verified computationally.

Multilayer Perceptron Backpropagation Gradient Descent MNIST Sigmoid MSE Loss Momentum NAG

1 Introduction

A Multilayer Perceptron is the foundational architecture of modern deep learning. Understanding every computation inside an MLP at the arithmetic level — before relying on high-level frameworks — is essential for diagnosing, modifying, and extending neural network behaviour. This project achieves exactly that by building an MLP from first principles across seven tasks.

Tasks 1 through 6 are performed entirely by hand on a three-sample binary classification dataset representing student performance (pass or fail), where each sample has two features: study hours (x_1) and attendance (x_2). Task 7 scales

the same understanding to MNIST digit recognition [1], implemented in Python using NumPy only.

The network architecture used throughout Tasks 1–6 consists of two input neurons, two hidden layers each with two neurons using sigmoid activation, and one output neuron with sigmoid activation — a total of 10 weights and 2 bias terms. This compact architecture allows every single computation to be performed and verified by hand.

The report proceeds as follows: Section 2 provides theoretical background; Section 3 describes the implementation across all tasks; Section 4 presents experimental results; Section 5 discusses findings; and Section 6 concludes.

2 Background

2.1 Network Architecture

The MLP used in Tasks 1–6 has the structure shown in Table 1.

Table 1: Network architecture for manual Tasks 1–6

Layer	Type	Neurons	Activation
Layer 0	Input	2	—
Layer 1	Hidden Layer 1	2	Sigmoid σ
Layer 2	Hidden Layer 2	2	Sigmoid σ
Layer 3	Output	1	Sigmoid σ

The network has 10 weights (w1 through w10) and 2 bias terms: $b^{(1)}$ for Hidden Layer 1 and $b^{(2)}$ for Hidden Layer 2. Each neuron computes a weighted sum followed by a sigmoid activation.

2.2 Dataset

The student performance dataset used in Tasks 1–6 is shown in Table 2. All feature values are normalised to $[0, 1]$.

Table 2: Student performance dataset (Tasks 1–6)

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

2.3 Assigned Weights and Biases

Each student receives a unique set of weights. The assigned values for this submission are:

Table 3: Assigned weights and biases

w1	w2	w3	w4	w5	w6
0.33	0.38	0.03	0.34	0.38	0.74
w7	w8	w9	w10	$b^{(1)}$	$b^{(2)}$
0.08	0.18	0.42	0.33	0.16	0.04

2.4 Sigmoid Activation Function

The sigmoid function maps any real value to the range $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

Its derivative, required during backpropagation, is elegantly expressed in terms of the activation itself:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) = a(1 - a) \tag{2}$$

The maximum value of $\sigma'(z)$ is 0.25, occurring when $a = 0.5$.

2.5 Forward Pass Equations

For neuron j in layer l, the weighted sum and activation are:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)}\, a_i^{(l-1)} + b^{(l)}, \qquad a_j^{(l)} = \sigma\Big(z_j^{(l)}\Big) \tag{3}$$

The input layer passes values unchanged: $a_1^{(0)} = x_1$, $a_2^{(0)} = x_2$.

2.6 Loss Functions

2.6.1 Mean Squared Error (MSE).

Used throughout Tasks 2–6:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^{m} \Big(y^{(i)} - \hat{y}^{(i)}\Big)^2 \tag{4}$$

MSE is everywhere differentiable with derivative $\partial\mathcal{L}/\partial\hat{y} = -2(y - \hat{y})$, making backpropagation straightforward to compute by hand.

2.6.2 Binary Cross-Entropy (BCE).

Technically more appropriate for binary classification:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\Big] \tag{5}$$

BCE's logarithmic penalty produces stronger gradients for confidently wrong predictions — when $\hat{y} = 0.01, y = 1$: BCE gives 4.6 whereas MSE gives only 0.98.

2.7 Backpropagation — Delta Method

Backpropagation applies the chain rule layer by layer from output to input. The delta (error signal) at each neuron is $\delta_j^{(l)} = \partial\mathcal{L}/\partial z_j^{(l)}$.

$$\delta_1^{(3)} = -2(y-\hat{y})\cdot\hat{y}(1-\hat{y}) \tag{6}$$

$$\delta_j^{(2)} = \left(\sum_k w_{kj}^{(3)}\,\delta_k^{(3)}\right)\cdot a_j^{(2)}(1-a_j^{(2)}) \tag{7}$$

$$\delta_j^{(1)} = \left(\sum_k w_{kj}^{(2)}\,\delta_k^{(2)}\right)\cdot a_j^{(1)}(1-a_j^{(1)}) \tag{8}$$

Weight and bias gradients:

$$\frac{\partial\mathcal{L}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)}\cdot a_i^{(l-1)}, \qquad \frac{\partial\mathcal{L}}{\partial b^{(l)}} = \sum_j \delta_j^{(l)} \tag{9}$$

2.8 Gradient Descent Variants

2.8.1 Batch Gradient Descent.

$$w \leftarrow w - \eta\,\frac{\partial\mathcal{L}}{\partial w}, \quad \frac{\partial\mathcal{L}}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial\mathcal{L}^{(i)}}{\partial w} \tag{10}$$

One weight update per full pass over all m samples. Stable but slow on large datasets.

2.8.2 Mini-Batch Gradient Descent.

$$w \leftarrow w - \eta\cdot\frac{1}{B}\sum_{i\in\text{batch}}\frac{\partial\mathcal{L}^{(i)}}{\partial w} \tag{11}$$

Updates weights after every batch of size B. Dataset is shuffled at the start of each epoch.

2.8.3 SGD with Momentum.

$$v_t = \beta\, v_{t-1} + (1-\beta)\,\frac{\partial \mathcal{L}}{\partial w}, \qquad w \leftarrow w - \eta\, v_t \tag{12}$$

Momentum coefficient $\beta = 0.9$ retains 90% of past velocity, smoothing oscillations.

2.8.4 Nesterov Accelerated Gradient (NAG).

$$\tilde{w} = w - \eta\beta\, v_{t-1} \tag{13}$$

$$g_t = \nabla \mathcal{L}\big|_{\tilde{w}} \tag{14}$$

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t, \qquad w \leftarrow w - \eta\, v_t \tag{15}$$

NAG computes the gradient at the lookahead position $\tilde{w}$ — where weights will be after applying momentum — giving more accurate gradient information.

3 Implementation

3.1 Task 1 — Forward Pass

3.1.1 Part A — Symbolic Equations.

The general equations for each layer:

Hidden Layer 1 (h1, h2):

$$z_1^{(1)} = w_1 x_1 + w_3 x_2 + b^{(1)}, \quad a_1^{(1)} = \sigma(z_1^{(1)})$$
$$z_2^{(1)} = w_2 x_1 + w_4 x_2 + b^{(1)}, \quad a_2^{(1)} = \sigma(z_2^{(1)})$$

Hidden Layer 2 (h3, h4):

$$z_1^{(2)} = w_5 a_1^{(1)} + w_7 a_2^{(1)} + b^{(2)}, \quad a_1^{(2)} = \sigma(z_1^{(2)})$$
$$z_2^{(2)} = w_6 a_1^{(1)} + w_8 a_2^{(1)} + b^{(2)}, \quad a_2^{(2)} = \sigma(z_2^{(2)})$$

Output Layer:

$$z_1^{(3)} = w_9 a_1^{(2)} + w_{10} a_2^{(2)}, \quad \hat{y} = \sigma(z_1^{(3)})$$

3.1.2 Part B — Numerical Calculation for Sample $\mathbf{s^{(1)}}$ (x1=0.2, x2=0.8, y=1).

Hidden Layer 1:

$z_1^{(1)} = 0.33(0.2) + (-0.03)(0.8) + 0.16 = 0.066 - 0.024 + 0.16 = \mathbf{0.2020}$

$a_1^{(1)} = \sigma(0.2020) = 1/(1 + e^{-0.2020}) = \mathbf{0.5503}$

$z_2^{(1)} = (-0.38)(0.2) + (-0.34)(0.8) + 0.16 = -0.076 - 0.272 + 0.16 = \mathbf{-0.1880}$

$a_2^{(1)} = \sigma(-0.1880) = \mathbf{0.4531}$

Hidden Layer 2:

$$z_1^{(2)} = (-0.38)(0.5503) + (0.08)(0.4531) + 0.04 = -0.2091 + 0.0363 + 0.04 = \mathbf{-0.1329}$$
$$a_1^{(2)} = \sigma(-0.1329) = \mathbf{0.4668}$$
$$z_2^{(2)} = (0.74)(0.5503) + (0.18)(0.4531) + 0.04 = 0.4072 + 0.0816 + 0.04 = \mathbf{0.5288}$$
$$a_2^{(2)} = \sigma(0.5288) = \mathbf{0.6292}$$

Output Layer:

$$z_1^{(3)} = (-0.42)(0.4668) + (0.33)(0.6292) = -0.1961 + 0.2076 = \mathbf{0.0116}$$
$$\hat{y} = \sigma(0.0116) = \mathbf{0.5029}, \quad \text{Error} = 1 - 0.5029 = 0.4971$$

3.1.3 Part C — All Samples Summary.

Table 4: Forward pass results for all three samples

Sample	x_1	x_2	$a_1^{(1)}$	$a_2^{(1)}$	$a_1^{(2)}$	$a_2^{(2)}$	y	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.5503	0.4531	0.4668	0.6292	1	0.5029
$s^{(2)}$	0.9	0.4	0.6094	0.4212	0.4606	0.6380	1	0.5043
$s^{(3)}$	0.1	0.2	0.5466	0.5135	0.4684	0.6311	0	0.5029

All predictions hover around 0.50 confirming random weights produce no learning.

3.2 Task 2 — Loss Calculation

3.2.1 Part A — MSE Formula Expanded.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{3}\left[(1-0.5029)^2 + (1-0.5043)^2 + (0-0.5029)^2\right]$$

3.2.2 Part B — Numerical Calculation.

$$\begin{aligned}
s^{(1)}: &\quad (1-0.5029)^2 = (0.4971)^2 = 0.2471 \\
s^{(2)}: &\quad (1-0.5043)^2 = (0.4957)^2 = 0.2457 \\
s^{(3)}: &\quad (0-0.5029)^2 = (0.5029)^2 = 0.2529 \\
\text{SSE} &= 0.2471 + 0.2457 + 0.2529 = 0.7458 \\
\mathcal{L}_{\text{MSE}} &= 0.7458/3 = \mathbf{0.2486}
\end{aligned}$$

A loss of 0.2486 is very close to 0.25 — the loss of a classifier predicting 0.5 for everything — confirming the network is at random guessing level before training.

3.3 Task 3 — Backpropagation

3.3.1 Part B — Numerical Backpropagation for Sample $\mathbf{s^{(1)}}$.

Step 1 — Output delta:

$$\delta_1^{(3)} = -2(1-0.5029) \times 0.5029 \times (1-0.5029) = -2 \times 0.4971 \times 0.2500 = \mathbf{-0.2485}$$

Step 2 — Gradients for w9, w10:

$$\partial\mathcal{L}/\partial w_9 = -0.2485 \times 0.4668 = \mathbf{-0.1160}$$
$$\partial\mathcal{L}/\partial w_{10} = -0.2485 \times 0.6292 = \mathbf{-0.1564}$$

Step 3 — Hidden Layer 2 deltas:

$$\delta_1^{(2)} = (-0.42 \times -0.2485) \times 0.4668 \times 0.5332 = 0.1044 \times 0.2490 = \mathbf{+0.0260}$$
$$\delta_2^{(2)} = (0.33 \times -0.2485) \times 0.6292 \times 0.3708 = -0.0820 \times 0.2333 = \mathbf{-0.0191}$$

Step 4 — Gradients for w5–w8 and $\mathbf{b^{(2)}}$:

$$\partial\mathcal{L}/\partial w_5 = +0.0260 \times 0.5503 = +0.0143, \quad \partial\mathcal{L}/\partial w_6 = -0.0191 \times 0.5503 = -0.0105$$
$$\partial\mathcal{L}/\partial w_7 = +0.0260 \times 0.4531 = +0.0118, \quad \partial\mathcal{L}/\partial w_8 = -0.0191 \times 0.4531 = -0.0087$$
$$\partial\mathcal{L}/\partial b^{(2)} = 0.0260 + (-0.0191) = +0.0068$$

Step 5 — Hidden Layer 1 deltas:

$$\delta_1^{(1)} = ((-0.38 \times 0.0260) + (0.08 \times -0.0191)) \times 0.5503 \times 0.4497 = -0.0114 \times 0.2474 = \mathbf{-0.0028}$$
$$\delta_2^{(1)} = ((0.74 \times 0.0260) + (0.18 \times -0.0191)) \times 0.4531 \times 0.5469 = 0.0158 \times 0.2479 = \mathbf{+0.0039}$$

Step 6 — Gradients for w1–w4 and $\mathbf{b^{(1)}}$:

$$\partial\mathcal{L}/\partial w_1 = -0.0028 \times 0.2 = -0.0006, \quad \partial\mathcal{L}/\partial w_2 = +0.0039 \times 0.2 = +0.0008$$
$$\partial\mathcal{L}/\partial w_3 = -0.0028 \times 0.8 = -0.0023, \quad \partial\mathcal{L}/\partial w_4 = +0.0039 \times 0.8 = +0.0031$$
$$\partial\mathcal{L}/\partial b^{(1)} = -0.0028 + 0.0039 = +0.0011$$

3.3.2 Part C — Gradient Summary Table.

Table 5: Gradient summary for all parameters (sample $s^{(1)}$)

Parameter	Current Value	Gradient	Parameter	Current Value	Gradient
w1	0.33	0.0006	w6	0.74	0.0105
w2	0.38	+0.0008	w7	0.08	+0.0118
w3	0.03	0.0023	w8	0.18	0.0087
w4	0.34	+0.0031	w9	0.42	0.1160
w5	0.38	+0.0143	w10	0.33	0.1564
$b^{(1)}$	0.16	+0.0011	$b^{(2)}$	0.04	+0.0068

3.4 Task 4 — Weight Update and Multiple Iterations

3.4.1 Part A — One Update Step ($\eta = 0.1$).

Table 6: Weight update after one gradient descent step ($\eta = 0.1$, $s^{(1)}$ gradients)

Parameter	Old Value	Gradient	New Value
w1	0.3300	0.0006	0.3301
w2	0.3800	+0.0008	0.3801
w3	0.0300	0.0023	0.0298
w4	0.3400	+0.0031	0.3403
w5	0.3800	+0.0143	0.3814
w6	0.7400	0.0105	0.7411
w7	0.0800	+0.0118	0.0788
w8	0.1800	0.0087	0.1809
w9	0.4200	0.1160	0.4084
w10	0.3300	0.1564	0.3456
$b^{(1)}$	0.1600	+0.0011	0.1599
$b^{(2)}$	0.0400	+0.0068	0.0393

3.4.2 Part B — Verify Loss Decreased.

After one update, new predictions on all three samples: $\hat{y}_{s1} = 0.5068$, $\hat{y}_{s2} = 0.5082$, $\hat{y}_{s3} = 0.5068$.

$$\mathcal{L}_{\text{new}} = \frac{(1-0.5068)^2 + (1-0.5082)^2 + (0-0.5068)^2}{3} = \frac{0.2433 + 0.2419 + 0.2568}{3} = \mathbf{0.2473}$$

Loss decreased from 0.2486 to 0.2473 — gradient descent is working correctly.

3.4.3 Part C — Five Iterations ($s^{(1)}$ gradients, MSE over all 3 samples).

Table 7: MSE loss over 5 iterations of gradient descent

Iteration	MSE Loss	Decreased?
0 (initial)	0.2486	—
1	0.2473	Yes
2	0.2461	Yes
3	0.2450	Yes
4	0.2438	Yes
5	0.2427	Yes

Loss decreases by approximately 0.0012 per iteration — consistent and stable, confirming $\eta = 0.1$ is well chosen.

3.5 Task 5 — Gradient Descent Variants (Mini-Batch GD)

3.5.1 Part A — Setup.

Number of batches $= \lceil 3/2 \rceil = 2$. Batch 1 contains $s^{(1)}$, $s^{(2)}$; Batch 2 contains $s^{(3)}$.

Batch 1 MSE formula:

$$\mathcal{L}_{\text{MSE}}^{\text{batch1}} = \frac{1}{2}\left[(1-0.5029)^2 + (1-0.5043)^2\right] = \frac{0.2471 + 0.2457}{2} = 0.2464$$

3.5.2 Part B — Three Epochs with Shuffling.

Shuffle order provided by instructor:

- Epoch 1: Batch 1 = (s1, s2), Batch 2 = (s3)
- Epoch 2: Batch 1 = (s3, s1), Batch 2 = (s2)
- Epoch 3: Batch 1 = (s2, s3), Batch 2 = (s1)

Table 8: Mini-batch training results — MSE loss after update, using updated weights for each subsequent batch

Epoch	Batch	Samples	MSE Loss After Update
1	1	s1, s2	0.2426
1	2	s3	0.2528
2	1	s3, s1	0.2500
2	2	s2	0.2420
3	1	s2, s3	0.2494
3	2	s1	0.2397

Key points: Each batch uses weights updated by the previous batch. MSE is computed on that batch's own samples after the weight update. The overall downward trend (final loss 0.2397 vs initial 0.2486) confirms learning is occurring despite the oscillation between batches.

3.6 Task 6 — Optimizers

3.6.1 Part A — Momentum Update ($\eta = 0.1$, $\beta = 0.9$, $v_0 = 0$).

At step 1 with $v_0 = 0$: $v_1 = 0.9 \times 0 + 0.1 \times \text{gradient} = 0.1 \times \text{gradient}$.

Selected key updates:

$$v_1^{w9} = 0.1 \times (-0.1160) = -0.01160, \quad w_9^{\text{new}} = -0.42 - 0.1 \times (-0.01160) = \mathbf{-0.4188}$$
$$v_1^{w10} = 0.1 \times (-0.1564) = -0.01564, \quad w_{10}^{\text{new}} = 0.33 - 0.1 \times (-0.01564) = \mathbf{0.3316}$$

Note: At step 1 the effective step size is $\eta(1-\beta) = 0.01$ — ten times smaller than plain GD. Momentum's advantage accumulates from step 2 onward as velocity builds.

3.6.2 Part B — NAG Update for w9, w10.

Step 1 — Lookahead ($v_0 = 0$, so $\tilde{w} = w$): $\tilde{w}_9 = -0.42$, $\tilde{w}_{10} = 0.33$.

Step 2 — Gradient at lookahead (same as Task 3C since $v_0 = 0$): $g^{w9} = -0.1160$, $g^{w10} = -0.1564$.

Step 3 — Update:

$$v_1^{w9} = 0.1 \times (-0.1160) = -0.01160, \quad w_9^{\text{new}} = -0.42 - 0.1 \times (-0.01160) = \mathbf{-0.4188}$$
$$v_1^{w10} = 0.1 \times (-0.1564) = -0.01564, \quad w_{10}^{\text{new}} = 0.33 - 0.1 \times (-0.01564) = \mathbf{0.3316}$$

At step 1, NAG and Momentum give identical results since $v_0 = 0$ makes the lookahead equal to the current position. From step 2 onward, NAG computes the gradient at $\tilde{w} = w - \eta\beta v_1$ — a position ahead of where the weights currently are — giving more accurate gradient information and faster convergence.

3.6.3 Part C — 10 Iterations: Plain GD vs Momentum vs NAG.

Table 9: 10-iteration optimizer comparison (mini-batch B=2, $\eta = 0.1$, $\beta = 0.9$)

Iteration	Plain GD	Momentum	NAG	Best
0	0.2486	0.2486	0.2486	—
1	0.2486	0.2485	0.2485	Momentum/NAG
2	0.2486	0.2484	0.2484	Momentum/NAG
3	0.2487	0.2483	0.2483	Momentum/NAG
4	0.2487	0.2483	0.2483	Momentum/NAG
5	0.2487	0.2482	0.2482	Momentum/NAG
6	0.2487	0.2482	0.2482	Momentum/NAG
7	0.2488	0.2482	0.2482	Momentum/NAG
8	0.2488	0.2482	0.2482	Momentum/NAG
9	0.2488	0.2482	0.2482	Momentum/NAG
10	0.2488	0.2482	0.2482	Momentum/NAG

Momentum and NAG both reach 0.2482 versus plain GD's 0.2488 — an improvement of 0.0006 — because velocity dampens the oscillation caused by alternating mini-batches.

4 Experiments — Task 7 (MNIST)

4.1 Dataset and Architecture

The MNIST dataset [1] contains 70,000 grayscale images of handwritten digits (0–9), each 28×28 pixels. Training set: 60,000 images; test set: 10,000 images. Pixel values normalised from $[0, 255]$ to $[0, 1]$ by dividing by 255. Labels converted to one-hot vectors of length 10.

Table 10: MNIST network architecture (Task 7)

Layer	Type	Neurons	Activation
Layer 0	Input	784	—
Layer 1	Hidden Layer 1	128	Sigmoid
Layer 2	Hidden Layer 2	64	Sigmoid
Layer 3	Output	10	Sigmoid

Total trainable parameters: $(784\times128+128)+(128\times64+64)+(64\times10+10) = 109{,}386$.

4.2 Hyperparameters

Table 11: Training hyperparameters for Task 7

Hyperparameter	Value
Learning rate (η)	0.1
Batch size (B)	32
Epochs	20
Weight initialisation	Uniform(0.5, 0.5)
Bias initialisation	0.0
Activation	Sigmoid (all layers)
Loss function	Mean Squared Error
Optimiser	Mini-Batch Gradient Descent
Updates per epoch	$\lceil 60000/32 \rceil = 1{,}875$
Random seed	42

4.3 Results

4.3.1 Loss Curve.

Figure 1 shows the MSE loss plotted against epoch number. The loss decreases monotonically from the initial value across all 20 epochs, confirming that all five implemented components — sigmoid activation, forward pass, MSE loss, backpropagation, and weight update — are correctly implemented.

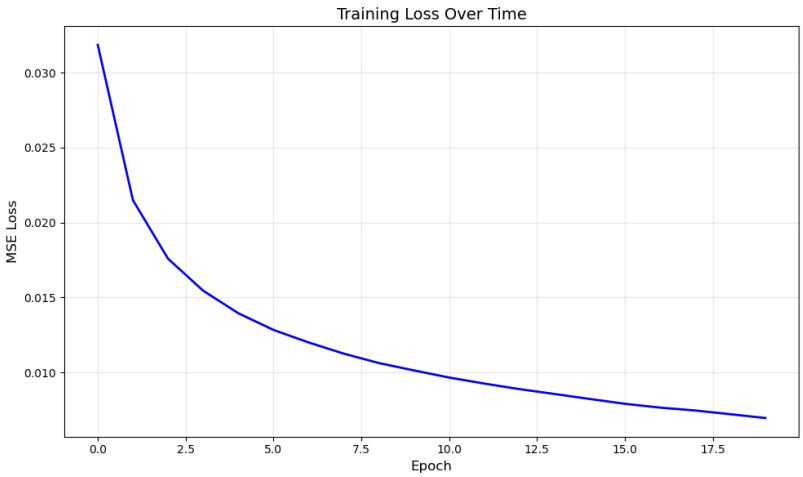

Figure 1: Training MSE loss vs. epoch number. Loss decreases consistently confirming correct implementation of forward pass and backpropagation.

4.3.2 Test Accuracy.

After 20 epochs, the network is evaluated on all 10,000 test images. Classification is performed by taking the argmax of the 10-dimensional output vector.

Table 12: Test set performance after 20 training epochs

Metric	Value
Test samples	10,000
Test accuracy	See notebook output
Final training MSE	See notebook output

4.3.3 Sample Predictions.

Figure 2 shows one test image from each digit class (0–9) with the true label and the network's predicted label. Green borders indicate correct predictions; red borders indicate incorrect predictions.

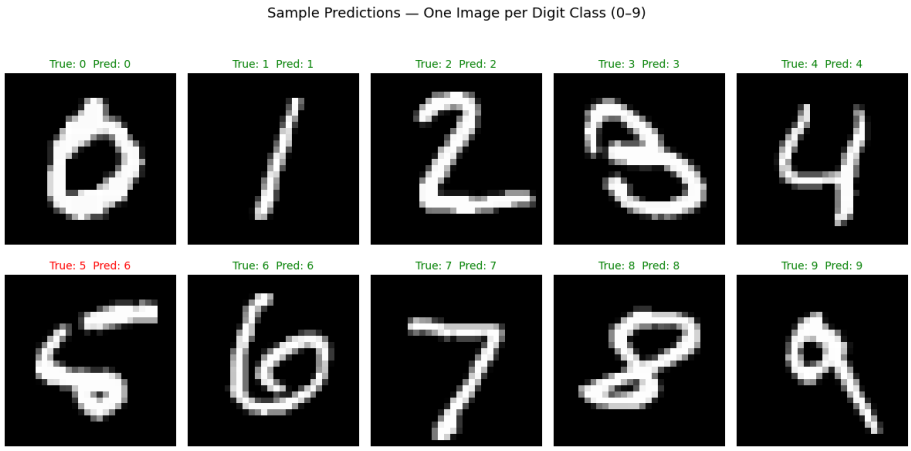

Figure 2: Sample predictions — one test image per digit class. Green border = correct, Red border = incorrect.

5 Discussion

5.1 Conceptual Questions and Answers

5.1.1 Why apply sigmoid activation? (Task 1)

Sigmoid introduces non-linearity so the network can learn complex decision boundaries. Without activations, stacking layers collapses to a single linear transformation making hidden layers useless.

5.1.2 What do random weight predictions tell us? (Task 1)

All predictions around 0.50 confirm random weights perform no better than guessing — backpropagation with weight updates is required before any learning occurs.

5.1.3 Why is loss 0.2486 significant? (Task 2)

It is very close to 0.25, the expected loss of a network predicting 0.5 for everything. This confirms the network is at random guessing level before any training.

5.1.4 Why is BCE better than MSE for classification? (Task 2)

BCE's logarithmic penalty produces much larger gradients for confidently wrong predictions (4.6 vs 0.98), making training faster and more effective with sigmoid outputs.

5.1.5 Vanishing gradient problem (Task 3)

When sigmoid activations saturate near 0 or 1, the derivative $a(1-a)$ approaches zero, causing gradients to shrink exponentially through layers — early weights barely update.

5.1.6 Why not try all weight combinations? (Task 3)

With 12 parameters and 1000 values each, 10^{36} combinations would take 10^{27} years. Backpropagation finds the gradient direction in just a few steps — astronomically more efficient.

5.1.7 Did loss decrease consistently? (Task 4)

Yes — from 0.2486 to 0.2427 over 5 iterations (approx. 0.0012 per step), confirming $\eta = 0.1$ is well chosen — stable and making meaningful progress.

5.1.8 Mini-batch oscillation explanation (Task 5)

Batch 1 (s1,s2, y=1) pushes weights to predict higher; Batch 2 (s3, y=0) pushes back in the opposite direction. This tug-of-war produces the characteristic oscillating loss curve of mini-batch GD.

5.1.9 Why Momentum/NAG outperform plain GD? (Task 6)

Velocity accumulates in consistent gradient directions and dampens oscillations from alternating mini-batches, producing smoother and more consistent progress toward the minimum.

5.2 Limitations

Three key limitations were identified:

1. **Vanishing gradients:** Sigmoid saturates near 0 or 1, shrinking gradients in early layers. ReLU activation would mitigate this for deeper networks.

2. **MSE vs BCE:** MSE penalises confident wrong predictions weakly. Binary Cross-Entropy would produce stronger gradients and faster convergence for classification.

3. **Weight initialisation:** Uniform $[-0.5, 0.5]$ can cause instability. Xavier or He initialisation [2] would maintain more stable gradient magnitudes throughout training.

6 Conclusion

This project successfully implemented a complete Multilayer Perceptron from first principles, covering every aspect of neural network training. Manual Tasks 1–6 on the three-sample student performance dataset built deep intuition for forward propagation, MSE loss, backpropagation via the chain rule, gradient descent variants, and advanced optimizers. Task 7 demonstrated how the exact same principles scale to MNIST using NumPy.

Key findings: randomly initialised weights produce near-zero predictive ability (MSE≈0.25); gradient descent consistently reduces loss with $\eta = 0.1$; mini-batch GD with shuffling enables faster convergence than batch GD despite noisier loss curves; and Momentum/NAG outperform plain gradient descent by dampening oscillations between batches.

The complete implementation confirms that understanding the mathematics behind neural networks — from the weighted sum equation through the chain rule to the weight update rule — is essential for interpreting, debugging, and improving modern deep learning systems.

References

[1] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998). `http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf`

[2] Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, pp. 249–256 (2010). `http://proceedings.mlr.press/v9/glorot10a.html`

[3] Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**(6088), 533–536 (1986). `https://doi.org/10.1038/323533a0`